Computer Networks Lab 8

Muhammad Saad Tahir 24k-0851 BCS-5B

I use Ubuntu 26.04 as my main personal OS hence the following settings are used here.

ALL Step:

```
muhammad-saad-tahir@MST:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 26.04.1 LTS
Release:        26.04
Codename:       resolute
muhammad-saad-tahir@MST:~$ uname -a
Linux MST 7.0.0-42-generic #42-Ubuntu SMP PREEMPT_DYNAMIC Thu Oct 15 08:06:44 UTC 2026 x86_64 GNU/Linux
muhammad-saad-tahir@MST:~$ python3 --version
g++ --version
cmake --version
Python 3.14.4
g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

cmake version 4.2.3
```

```
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 18
muhammad-saad-tahir@MST:~$ g++ --version
cmake --version
ninja --version
git --version
python3 -c "import numpy, matplotlib; print('NumPy and Matplotlib installed successfully')"
g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

cmake version 4.2.3

CMake suite maintained and supported by Kitware (kitware.com/cmake).
1.13.2
git version 2.53.0
NumPy and Matplotlib installed successfully
muhammad-saad-tahir@MST:~$
```

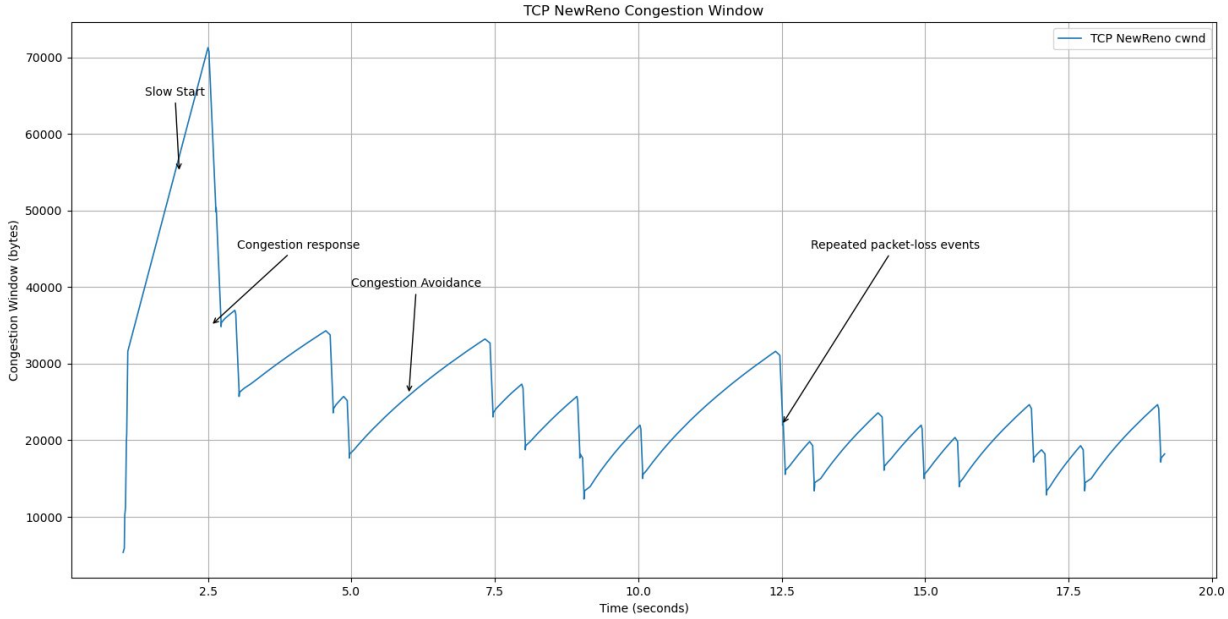
```
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ ./ns3 build
[0/2] Re-checking globbed directories...
[1623/1624] Linking CXX executable /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/build/utils/ns3-dev-
Finished executing the following commands:
/usr/bin/cmake --build /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/cmake-cache -j 7
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ ./ns3 run hello-simulator
[0/2] Re-checking globbed directories...
ninja: no work to do.
Hello Simulator
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ nano scratch/first.cc
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ ./ns3 run scratch/first
-- Using default output directory /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/build
-- Proceeding without cmake-format
-- Some or all of the gtk libraries were not found. (missing: GTK3_GTK_LIBRARY GTK3_GTK_INCLUDE_DIR GTK3_GDK_
-- Python: development libraries were not found
-- Boost was not found
```

```
-- ---- Summary of ns-3 settings:
Build profile                 : default
Build directory               : /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/build
Build with runtime asserts    : ON
Build with runtime logging    : ON
Build version embedding       : OFF (not requested)
BRITE Integration             : OFF (Missing headers: "Brite.h" and missing libraries: "brite")
DES Metrics event collection  : OFF (not requested)
DPDK NetDevice                : OFF (not requested)
Emulation FdNetDevice         : ON
Examples                      : ON
File descriptor NetDevice     : ON
GNU Scientific Library (GSL)  : OFF (GSL was not found)
GtkConfigStore                : OFF (GTK3 was not found)
LibXml2 support               : ON
MPI Support                   : OFF (not requested)
ns-3 Click Integration        : OFF (Missing headers: "simclick.h" and missing libraries: "nsclick click")
ns-3 OpenFlow Integration     : OFF (Missing headers: "openflow.h" and missing libraries: "openflow")
Netmap emulation FdNetDevice  : OFF (missing dependency)
PyViz visualizer              : OFF (Python Bindings are disabled)
Python Bindings               : OFF (not requested)
SQLite support                : OFF (missing dependency)
Eigen3 support                : OFF (Eigen was not found)
Tap Bridge                    : ON
Tap FdNetDevice               : ON
Tests                         : OFF (not requested)
```

```
-- Configuring done (1.3s)
-- Generating done (0.5s)
-- Build files have been written to: /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/cmake-cache
[0/2] Re-checking globbed directories...
[2/3] Linking CXX executable /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/build/scratch/ns3-dev-first
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ nano scratch/tcp-congestion.cc
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ nano scratch/tcp-congestion.cc
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ rm -f cwnd.dat
./ns3 run scratch/tcp-congestion
-- Using default output directory /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/build
-- Proceeding without cmake-format
```

```
-- Configuring done (1.2s)
-- Generating done (0.5s)
-- Build files have been written to: /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/cmake-cache
[0/2] Re-checking globbed directories...
[2/3] Linking CXX executable /home/muhammad-saad-tahir/Projects/ns3-lab08/ns-3-dev/build/scratch/ns3-de
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ ls -lh cwnd.dat
-rw-rw-r-- 1 muhammad-saad-tahir muhammad-saad-tahir 17K Nov  6 23:48 cwnd.dat
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ head cwnd.dat
1.02019 5360
1.0413 5896
1.04319 6968
1.04508 8040
1.04697 9112
1.04885 10184
1.06233 11256
1.06422 12328
1.06611 13400
1.068 14472
muhammad-saad-tahir@MST:~/Projects/ns3-lab08/ns-3-dev$ wc -l cwnd.dat
1240 cwnd.dat
```

Figure: It is generated by python code provided in github link.



TCP NewReno Congestion Window Analysis

1. The graph illustrates TCP NewReno congestion-window behaviour over a 20-second simulation.
2. Initially, the congestion window increases rapidly during the Slow Start phase.
3. This allows the sender to increase its transmission rate as acknowledgements arrive.
4. The congestion window reaches approximately 71 KB around 2.5 seconds.
5. Packet loss triggers TCP's congestion response.
6. The congestion window is reduced to control network congestion.
7. During Congestion Avoidance, the window increases gradually.
8. This gradual increase allows TCP to probe available network capacity.
9. Further packet-loss events cause additional congestion-window reductions.
10. NewReno repeatedly alternates between increasing and reducing its congestion window.
11. These fluctuations demonstrate TCP's congestion-control mechanism.
12. Overall, the simulation demonstrates how TCP NewReno adapts its sending rate to network conditions.